i_03 PIPELINE_REGISTRY移行設計

目的

task_router.pyのPIPELINE_REGISTRY（フラット辞書）を、BASE_PIPELINES + INDUSTRY_PIPELINES の2層構造に移行する。

現在の構造

```
PIPELINE_REGISTRY: dict[str, str] = {
    "construction/estimation":
"workers.bpo.construction.pipelines.estimation_pipeline.run_estimation_pipeline",
    "sales/outreach":
"workers.bpo.sales.marketing.outreach_pipeline.run_outreach_pipeline",
    "common/expense":
"workers.bpo.common.pipelines.expense_pipeline.run_expense_pipeline",
    "backoffice/invoice_issue":
"workers.bpo.common.pipelines.invoice_issue_pipeline.run_invoice_issue_pipeline",
    ...
}
```

新しい構造

```
# Layer A: 全テナントで自動有効化
BASE_PIPELINES: dict[str, str] = {
    # 営業SFA/CRM/CS
    "sales/outreach":
"workers.base.sales.marketing.outreach_pipeline.run_outreach_pipeline",
    "sales/lead_qualification":
"workers.base.sales.sfa.lead_qualification_pipeline.run_lead_qualification_pipeline",
    ...（12本）
    # バックオフィス
    "backoffice/expense":
"workers.base.backoffice.accounting.expense_pipeline.run_expense_pipeline",
    "backoffice/invoice_issue":
"workers.base.backoffice.accounting.invoice_issue_pipeline.run_invoice_issue_pipeline",
    ...（24本）
}

# Layer B: テナントの業界選択で有効化
INDUSTRY_PIPELINES: dict[str, dict[str, str]] = {
    "construction": {
        "estimation":
"workers.industry.construction.pipelines.estimation_pipeline.run_estimation_pipeline",
        "billing":
"workers.industry.construction.pipelines.billing_pipeline.run_billing_pipeline",
        ...（8本）
    },
    "manufacturing": {
        "quoting":
"workers.industry.manufacturing.pipelines.quoting_pipeline.run_quoting_pipeline",
        ...（8本）
    },
    ...
}
```

パイプラインキーの変更

旧キー	新キー	層
`common/expense`	`backoffice/expense`	BASE
`common/payroll`	`backoffice/payroll`	BASE
`common/attendance`	`backoffice/attendance`	BASE
`common/contract`	`backoffice/contract`	BASE
`common/vendor`	`backoffice/vendor`	BASE
`common/admin_reminder`	`backoffice/admin_reminder`	BASE
`construction/estimation`	`construction/estimation`	INDUSTRY
`manufacturing/quoting`	`manufacturing/quoting`	INDUSTRY
`sales/outreach`	`sales/outreach`	BASE（変更なし）
`backoffice/invoice_issue`	`backoffice/invoice_issue`	BASE（変更なし）

route_and_execute の変更

```
async def route_and_execute(task: BPOTask, ...):
    pipeline_key = task.pipeline

    # Step 1: BASE_PIPELINES を先にチェック
    if pipeline_key in BASE_PIPELINES:
        module_path = BASE_PIPELINES[pipeline_key]

    # Step 2: INDUSTRY_PIPELINES をチェック
    elif "/" in pipeline_key:
        industry, pipeline_name = pipeline_key.split("/", 1)
        # テナントの業界と一致するか検証
        tenant_industry = await get_tenant_industry(task.company_id)
        if tenant_industry != industry:
            return PipelineResult(success=False, error="業界不一致")
        if industry in INDUSTRY_PIPELINES and pipeline_name in
INDUSTRY_PIPELINES[industry]:
            module_path = INDUSTRY_PIPELINES[industry][pipeline_name]
        else:
            return PipelineResult(success=False, error="未登録パイプライン")

    else:
        return PipelineResult(success=False, error="不正なキー形式")

    # 以下は現在と同じ（動的ロード→実行）
```

ドメインエイリアスの扱い

現在のエイリアス（marketing/outreach → sales/outreach 等）は廃止するか維持するかの判断:
- **推奨: 廃止**。フロントエンドは正規キーのみ使用。
- 移行期間として3ヶ月間は旧キーを新キーにリダイレクトする互換マップを用意:

```
LEGACY_KEY_MAP = {
    "common/expense": "backoffice/expense",
    "marketing/outreach": "sales/outreach",
    "sfa/lead_qualification": "sales/lead_qualification",
    ...
}
```

移行手順

1. 新構造を定義（BASE_PIPELINES + INDUSTRY_PIPELINES）

2. LEGACY_KEY_MAP を追加（旧キー→新キー変換）
3. route_and_execute に2層チェック + レガシー変換を実装
4. スケジュールトリガー・イベントトリガーのキーを新キーに更新
5. フロントエンドのAPI呼び出しを新キーに更新
6. 3ヶ月後: LEGACY_KEY_MAP を削除